Blueprint Técnico

Este documento entrega a quem vai abrir o repositório tudo o que é preciso para colocar a fase 1 no ar: a arquitetura, o formato da sessão viva, a máquina de estados do mestre, os contratos com o emulador e com o serviço de linguagem, a extensão do canal do emulador e o roadmap em cards, na ordem em que devem ser executados.

34	12	26	2
casos de uso da fase 1 cobertos	tabelas na sessão viva	cards na ordem de execução	provas executadas no emulador do cliente

Para quem escrevemos isto. Para a pessoa que vai criar o repositório e precisa saber exatamente o que construir primeiro. Cada capítulo responde a uma pergunta prática: qual é o escopo real, como a sessão é guardada, como o mestre muda de estado, qual é o contrato de cada mensagem, o que muda no emulador e qual card pegar em seguida. Nada aqui é sugestão de arquitetura para discutir depois: é o que vamos construir.

A regra que organiza todo o documento. O serviço de linguagem devolve intenção; só a plataforma aciona o emulador. Toda ação passa pelo catálogo antes e pelo registro depois. A sessão viva é só de acréscimo: nada é editado ou apagado. Se um card contrariar isso, o card está errado.

PROVADO NO EMULADOR DO CLIENTE, ANTES DE ESCREVER

- **Slot reservado.** Pelo canal oficial, salvar estado no slot 90 respondeu na hora; o arquivo apareceu em 0,3 s e ficou estável em 0,9 s, com 4,4 MB; recarregar o slot 90 funcionou e o jogo seguiu rodando. O emulador não valida número de slot e sua rotina de limpeza só varre os slots de 1 a 10: os slots 90 a 99 ficam fora do alcance do jogador.
- **Captura.** A captura por região de tela trouxe a janela que estava por cima do jogo. A captura pela janela, com conteúdo completo, trouxe o jogo mesmo coberto, com a renderização D3D12, em 21 ms. Reduzida a 640 pixels de largura e comprimida, a imagem ficou em 46 KB.

O QUE JÁ EXISTE NA MÁQUINA DO CLIENTE

- Emulador 2.8.2 portátil, com canal oficial ligado na porta 28011, BIOS e o jogo de abertura instalados e identificados pelo canal.
- Python 3.13 com tkinter, pillow, numpy, pydantic, httpx; captura e acesso a janelas instalados nesta rodada.
- Build Tools 2022 com a carga de trabalho C++, MSVC 14.44, SDK 10.0.26100, MSBuild, CMake e Ninja.
- Dependências do emulador para Windows publicadas pelo projeto em pacote pré-compilado de 164 MB.
- Disco: 87 GB livres após limpeza feita pelo cliente; a compilação do emulador cabe com folga.

Versão 1.1 · outubro de 2026 · emulador PCSX2, versão própria a partir da etiqueta corrente · Resident Evil 4 (SLUS-21134, versão 1.01) · acompanha o Mapa do Projeto (revisão 7) e os Casos de Uso (entrega 7). A 1.1 acrescentou a camada de cena (fase 4) ao capítulo 6 e a etapa S7 ao roadmap; a 1.2 acrescenta o perfil de memória e a descoberta de endereços ao capítulo 6 e a etapa S8 (fase 2 adiantada, DC-22); a 1.3, de 29 de outubro, registra o que S8-01 e S8-02 viraram na prática (vigia de memória e engenharia reversa do código; chamada remota, personagens e módulos carregados pelo mestre) e acrescenta a etapa S9, o mapa completo do jogo de abertura, que passa na frente das ações de mundo (DC-23); o resto é a 1.0 aprovada

CAPÍTULO 1

O escopo real da primeira entrega

A fase 1 tem 34 casos de uso em sete módulos. Nenhum deles funciona sozinho: o mestre só existe com a ponte, o olho, o catálogo, o overlay e o registro juntos. Vamos tratar isso como uma entrega só, em duas metades: primeiro o esqueleto jogável em modo reduzido, depois a versão própria do emulador e a robustez.

O QUE O JOGADOR SENTE

Módulos C, D, E · 15 casos
mestre decide, alavancas agem, overlay mostra, jogador reverte ou pausa

O QUE SUSTENTA

Módulo A · 7 casos na fase 1
canal oficial, identificação, slots reservados, pausa e velocidade

Módulo B · 4 casos
captura pela janela, situação em formato fechado, só o que mudou

Módulos F, G · 8 casos
sessão viva, custo, falhas, bancada

Como chegamos ao número. Dos 46 casos do índice, 34 são da fase 1. O caso A08, ler e escrever memória, é fase 2 e fica de fora junto com H, I e J. A primeira metade entregável, o esqueleto, cobre 21 casos em modo reduzido: tudo menos pausa e velocidade (A03, A04, parte de D01) e a robustez de G.

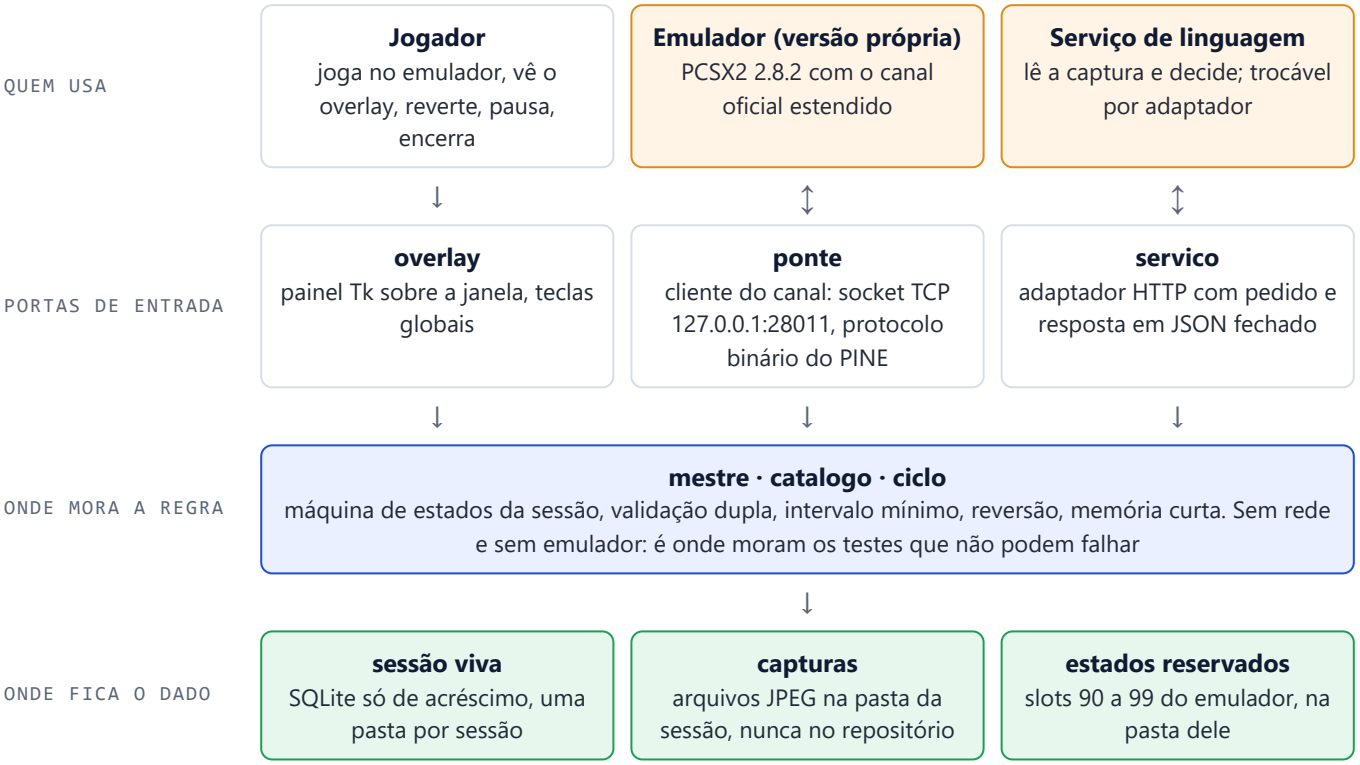
METADE	O QUE O CLIENTE JOGA NO FIM	CASOS COBERTOS	PRECISA DA VERSÃO PRÓPRIA DO EMULADOR?
1 · Esqueleto	Uma sessão inteira com o mestre vendo a tela, decidindo, salvando e recarregando estado em slots reservados, com overlay, reversão, pausa do mestre e registro completo.	A01 A02 A05 A06 A07 · B01 B02 B03 B04 · C01 C02 C03 C04 C05 · D01 D02 D03 D04 D05 · E01 E02 E03 E04 · F01 F02	Não. Roda no emulador original, em modo reduzido, como a ficha A07 prevê.
2 · Versão própria e robustez	As mesmas sessões com pausa dramática e câmera lenta, retomada após queda, custo com teto, exportação e a bancada no CI.	A03 A04 · E05 · F03 F04 · G01 G02 G03 G04	Sim, para A03 e A04. O resto não.

O que fica de fora, de propósito. Perfil de memória, ações de mundo, fontes públicas de mapas, perfis compartilháveis, entidades e controle virtual. Tudo isso está desenhado nos módulos H, I e J e entra depois que o cliente jogar a fase 1. A sessão viva e o catálogo deste documento já foram modelados para receber ações de mundo sem migração destrutiva: uma ação de mundo é uma alavanca com parâmetros, igual às universais.

CAPÍTULO 2

A arquitetura que vamos construir

São três processos e um lugar onde o dado mora. O mestre é um programa só, em Python, que fala com o emulador por um socket local e com o serviço de linguagem pela rede. O emulador é a versão própria do PCSX2, que só estende o servidor do canal oficial. Nada conversa com o emulador sem passar pelo catálogo.



Os pacotes do repositório do mestre

PONTE
Cliente do canal oficial: conectar, sondar status, identificar, ler memória, salvar e recarregar slot, e os comandos estendidos quando existirem. Confirma salvamento pelo arquivo. Nenhuma regra de negócio.

OLHO
Captura pela janela, recorte do overlay, redução, diferença entre capturas, pedido de situação ao serviço e validação do formato fechado.

MESTRE
O pedido de decisão, a memória curta, a validação dupla, o "nada" como decisão e o motivo em uma frase. Testável com respostas gravadas.

CATALOGO
Alavancas, faixas, disponibilidade, tons e intensidades, versão do catálogo, intervalo mínimo e a execução pelo caso de uso de cada alavanca.

SESSAO
O ciclo observar, decidir, executar; a máquina de estados; o overlay; abrir, retomar, encerrar; teclas globais.

REGISTRO
A sessão viva em SQLite, só de acréscimo; métricas; exportação; reprodução.

SERVICOS

Adaptadores do serviço de linguagem, um por provedor, atrás de uma interface única: descrever(captura, estado) e decidir(situação, estado, catálogo, memória).

SIMULADOR

Um servidor falso do canal oficial, em Python, que responde aos mesmos comandos com um jogo de mentira. É o que roda no CI e no notebook sem emulador.

BANCADA

Os casos sintéticos, o corredor que roda cada caso contra o serviço ou contra respostas gravadas, e o relatório comparável entre rodadas.

ESCOLHA	O QUE USAMOS	POR QUÊ
Linguagem	Python 3.13, já instalado na máquina do cliente	O protótipo é em Python; captura, janelas e overlay têm biblioteca madura; o esqueleto nasce rápido. O mestre é programa separado do emulador, com licença própria (DC-14).
Ponte	Socket TCP local com o protocolo binário do canal oficial, cliente próprio de 80 linhas	Sem dependência externa; o protocolo tem 16 comandos e cabe num arquivo. O simulador implementa o mesmo protocolo do outro lado.
Captura	Captura pela janela com conteúdo completo, pela API do sistema, recortando a área do cliente	Provado: pega o jogo mesmo com outra janela por cima, em 21 ms, com D3D12. Captura por região de tela foi reprovada na prova.
Overlay	Tkinter, janela sem borda, sempre por cima, arrastável, fora da área capturada	Já provado no protótipo; vem com o Python; sem dependência nova. Teclas globais por gancho do sistema.
Sessão viva	SQLite em modo WAL, um arquivo por sessão, mais a pasta de capturas	Transação, consulta e reprodução sem servidor; só de acréscimo por convenção e por gatilho que recusa UPDATE e DELETE.
Serviço de linguagem	Interface única com adaptador por provedor; o primeiro adaptador é o do provedor cujo acesso o cliente já tem	A escolha final é da bancada (G04), por custo, latência e aprovação por regra, nunca por preferência. Um decisor tipado entra como segundo adaptador para comparação.
Versão própria do emulador	Repositório derivado do PCSX2 2.8.2, mudanças só em PINE.cpp e no cabeçalho dele; compilação com MSBuild e as dependências pré-compiladas do projeto	Mantém o diff pequeno para acompanhar o original e para propor de volta (DC-13). A medição do custo de compilar é o primeiro card da etapa S4.
Testes	pytest; integração contínua no GitHub com o simulador; sem emulador e sem serviço no CI	O CI prova plataforma e catálogo; o serviço é provado pela bancada, sob demanda, com custo informado.

Por que o mestre é um processo e não um plugin do emulador. O emulador é C++ sob GPL e lança versão toda semana; o mestre muda todo dia e tem licença própria. Separados por um socket local, cada um evolui no seu ritmo, o emulador pode ser trocado pela versão original sem o mestre saber (modo reduzido), e o simulador substitui o emulador nos testes sem nenhuma condição especial no código.

CAPÍTULO 3

A sessão viva, tabela por tabela

São 12 tabelas num arquivo SQLite por sessão: três de contexto, seis de registro do ciclo, duas de apoio e uma de aceite. Antes do primeiro CREATE TABLE, cinco decisões que valem para todas elas.

SÓ ACRÉSCIMO

Nenhuma linha é alterada ou apagada depois de gravada. Correção entra como linha nova apontando para a anterior. Gatilhos recusam UPDATE e DELETE nas tabelas de registro.

TEMPO É DUPLO

Toda linha tem o relógio do sistema em UTC, para ler, e o contador monotônico da sessão em milissegundos, para medir. Intervalos e latências usam só o contador; o relógio nunca entra em conta (D04).

ESTADO É ENUMERAÇÃO NO BANCO

Fase da cena, alavanca, classe de falha e origem da decisão têm lista fechada conferida por CHECK. Gravar valor fora da lista é erro, não um defeito que se descobre depois.

A CAPTURA É ARQUIVO, A LINHA É REFERÊNCIA

A imagem fica em JPEG na pasta da sessão, nomeada pelo número do ciclo; a tabela guarda caminho, tamanho e impressão digital. O banco fica pequeno e a pasta pode ser apagada sem corromper o registro.

Quinta decisão: toda intervenção aponta para a decisão que a originou, e toda decisão para o ciclo. A cadeia ciclo, captura, situação, decisão, intervenção é o que a reprodução (F03) percorre e o que a regra RN-01 exige. Uma intervenção sem decisão é impossível de gravar: a chave estrangeira recusa.

ESQUEMA COMPLETO, PRONTO PARA RODAR · MIGRAÇÃO 0001

```
-- Roda uma vez por sessão, ao criar o arquivo. PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON.
create table sessoes (
  id              text primary key,           -- identificador ordenável pelo tempo
  aberta_em_utc   text not null,
  fechada_em_utc  text,                       -- nulo = sessão sem fim (E05 retoma)
  jogo_titulo     text not null,
  jogo_serial     text not null,              -- SLUS-21134
  jogo_crc        text not null,              -- 013e349d
  jogo_versao     text,
  emulador_versao text not null,
  modo            text not null check (modo in ('completo','reduzido')),
  tom             text not null,
  intensidade     text not null,
  servico         text not null,              -- nome do adaptador em uso
  aceite_versao   text not null,              -- versão do texto do aviso aceito (E02)
  config_json     text not null
);
```

```
create table catalogo_versoes (
  id          integer primary key,
  sessao_id   text not null references sessoes (id),
  versao      integer not null,
  criado_ms   integer not null,               -- contador da sessão
  motivo      text not null,                  -- abertura, alavanca indisponível, emulador reiniciado
  catalogo_json text not null,                -- alavancas, faixas, disponibilidade, motivos padrão
  unique (sessao_id, versao)
);
```

```
create table ciclos (
  id            integer primary key,
  sessao_id     text not null references sessoes (id),
  numero        integer not null,
  inicio_ms     integer not null,
  fim_ms        integer,
  status_emu    text not null check (status_emu in ('rodando','pausado','desligado','sem_canal')),
  estado_mestre text not null,                -- estado da máquina do capítulo 4 no início do ciclo
  lacuna        text,                          -- nulo, ou o motivo de não haver observação
  unique (sessao_id, numero)
);
```

```
create table capturas (
  id          integer primary key,
  ciclo_id    integer not null references ciclos (id),
  arquivo     text not null,                    -- capturas/000123.jpg, relativo à pasta da sessão
  largura     integer not null,
  altura      integer not null,
  bytes       integer not null,
  digest      text not null,                    -- impressão digital da imagem reduzida
  diferenca   real,                             -- medida contra a última enviada (B03)
  enviada     integer not null check (enviada in (0,1)),
  motivo_envio text                             -- mudança, intervalo vencido, releitura
);
```

```
create table situacoes (
  id          integer primary key,
  captura_id  integer not null references capturas (id),
  descricao   text not null,
  fase        text not null check (fase in
('exploracao','combate','menu','video','carregamento','morte','sistema')),
  aperto      integer not null check (aperto between 0 and 4),
  confianca   real not null check (confianca between 0 and 1),
  suspeita    text,                             -- nulo, ou por que contradiz o estado do emulador (B02
E3)
  latencia_ms integer not null,
  resposta_bruta text not null                  -- o JSON como veio, para a bancada
);
```

```
create table decisoes (
  id          integer primary key,
  ciclo_id    integer not null references ciclos (id),
  situacao_id integer references situacoes (id),  -- nulo quando a decisão é "nada" por falta de situação
  catalogo_versao integer not null,
  alavanca    text not null,                    -- 'nada' ou uma do catálogo
  parametros_json text not null,
  motivo      text not null,
  motivo_origem text not null check (motivo_origem in ('servico','padrao','ausente')),
  origem      text not null check (origem in ('servico','descarte','ritmo','bloqueio','falha')),
  descarte_motivo text,                         -- fora do catálogo, fora da faixa, regra de mesa,
situação velha
  pedido_bruto    text not null,
  resposta_bruta  text,
  latencia_ms integer
);
```

```
create table intervencoes (
  id            integer primary key,
  decisao_id    integer not null references decisoes (id),
  alavanca      text not null,
  parametros_json text not null,
  origem        text not null check (origem in ('mestre','reversao_jogador','encerramento')),
  estado_antes_json  text not null,            -- status, velocidade, slot mais recente, situação
  comando_ms    integer not null,
  confirmado_ms integer,                       -- nulo = não confirmado
  estado_depois_json text,
  resultado     text not null check (resultado in
('confirmada','falhou','descartada','cancelada_jogador')),
  fim_ms        integer,                       -- para ações com duração: quando voltou ao normal
  reversivel    integer not null check (reversivel in (0,1)),
  revertida_por integer references intervencoes (id),
  caminho_volta_json text                      -- ex.: slot de volta da recarga (A06)
);
```

```
create table estados_reservados (
  id          integer primary key,
  sessao_id   text not null references sessoes (id),
  slot        integer not null check (slot between 90 and 99),
  salvo_ms    integer not null,
  confirmado_ms integer,                       -- tamanho estável em duas leituras (LC-11)
  arquivo     text not null,
  bytes       integer,
  situacao_id integer references situacoes (id), -- contexto para a recarga
  motivo      text not null,                   -- periodico, antes_recarga, fim_sessao
  sobrescrito_ms integer,                      -- quando a fila avançou por cima dele
  suspeito    integer not null default 0
);
```

```
create table falhas (
  id          integer primary key,
  ciclo_id    integer references ciclos (id),
  ocorrido_ms integer not null,
  origem      text not null check (origem in ('servico','emulador','captura','registro')),
  classe      text not null,                   -- lista fechada do capítulo 5
  detalhe     text not null,
  visivel     integer not null check (visivel in (0,1)),  -- apareceu no overlay (RN-10)
  resolvida_ms integer
);
```

```
create table metricas (
  id          integer primary key,
  ciclo_id    integer not null references ciclos (id),
  etapa       text not null check (etapa in ('captura','descrever','decidir','executar','confirmar')),
  latencia_ms integer not null,
  tokens_entrada integer,
  tokens_saida   integer,
  custo_estimado real                          -- na moeda da tabela de preço configurada
);
```

```
create table marcos (
  id          integer primary key,
  ciclo_id    integer not null references ciclos (id),
  tipo        text not null check (tipo in
('morte','mudanca_fase','reversao','pausa_mestre','retomada','lacuna')),
  detalhe     text
);
```

```
create table aceites (
  id          integer primary key,
  aceito_em_utc text not null,
  aviso_versao text not null,
  servico     text not null,
  aceito      integer not null check (aceito in (0,1))
);
```

OS GATILHOS QUE SEGURAM A REGRA DE SÓ ACRÉSCIMO

```
-- Um por tabela de registro. Tentar alterar ou apagar devolve erro e nada muda.
create trigger ciclos_sem_update before update on ciclos
  when old.fim_ms is not null begin select raise(ABORT, 'registro só de acréscimo'); end;
create trigger ciclos_sem_delete before delete on ciclos begin select raise(ABORT, 'registro só de
acréscimo'); end;
-- O mesmo par para capturas, situacoes, decisoes, intervencoes, falhas, metricas e marcos.
-- Exceção explícita: ciclos.fim_ms,
intervencoes.confirmado_ms/estado_depois/resultado/fim_ms/revertida_por
-- e estados_reservados.confirmado_ms/sobrescrito_ms/suspeito são preenchidos uma única vez, de nulo para
valor.
-- O gatilho de cada uma dessas tabelas permite UPDATE só quando o valor antigo da coluna é nulo.
```

Retenção: o que apagamos e quando

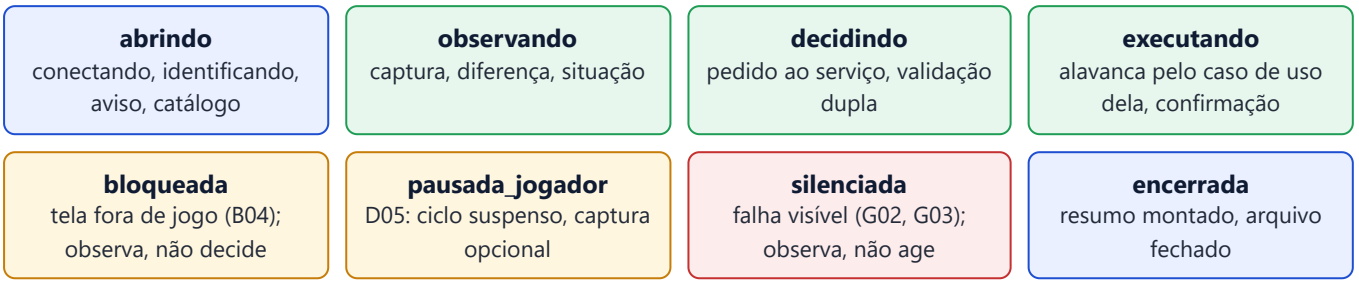
O QUÊ	RETENÇÃO	POR QUÊ
Arquivo da sessão (SQLite)	Permanente, na pasta do jogador	É a prova de tudo e a matéria da bancada. Pequeno: texto e números.
Pasta de capturas	Configurável; padrão 30 dias; o jogador pode apagar a qualquer hora	São imagens do jogo dele. O registro continua íntegro sem elas: a reprodução mostra "captura apagada".
Estados reservados (slots 90 a 99)	Fila de 10 por jogo; o mais antigo é sobrescrito; apagados ao encerrar se o jogador pedir	4,4 MB cada no jogo de abertura; 10 slots são o teto do que a reversão precisa.
Respostas brutas do serviço	Junto com a sessão	Sem elas a bancada não reproduz decisões antigas.

Este esquema foi executado, não só escrito. Todo o SQL acima roda num SQLite limpo antes de entrar no repositório, como primeiro passo do card S0-02, e os gatilhos são provados com as três tentativas que o card lista: alterar um ciclo fechado, apagar uma decisão, gravar fase fora da lista. As três têm de falhar.

CAPÍTULO 4

A máquina de estados da sessão

Esta é a peça central do mestre. Uma função pura recebe o estado atual e um evento, e devolve o novo estado e os efeitos. Nenhum outro lugar do código muda o estado da sessão. Se alguém precisar mudar, muda aqui, com teste.



Os oito estados. Verde é o ciclo normal, que gira observando, decidindo, executando, observando. Âmbar são as paradas que o jogo ou o jogador impõem. Vermelho é a falha, que silencia o mestre sem parar o registro. A regra de ouro: só de **executando** sai um comando para o emulador, e só se o evento foi **decisao_validada**.

A ASSINATURA, NO PACOTE SESSAO

```
Estado = 'abrindo' | 'observando' | 'decidindo' | 'executando' | 'bloqueada' | 'pausada_jogador' |
'silenciada' | 'encerrada'

Evento = 'pronto' | 'captura_sem_mudanca' | 'situacao_ok' | 'situacao_fora_de_jogo' | 'situacao_invalida'
       | 'decisao_validada' | 'decisao_nada' | 'decisao_descartada' | 'intervalo_nao_venceu'
       | 'execucao_confirmada' | 'execucao_falhou' | 'situacao_mudou'
       | 'jogador_pausou' | 'jogador_retomou' | 'jogador_reverteu' | 'jogador_encerrou'
       | 'falha_servico' | 'falha_emulador' | 'falha_registro' | 'recuperou' | 'cena_voltou'

# Sem rede, sem emulador, sem relógio: só estado e evento. Por isso é testável.
def transicao(atual: Estado, evento: Evento) -> Resultado:
    # Resultado = (proximo: Estado, efeitos: list[Efeito]) ou erro 'transicao_invalida'
```

A tabela de transições, inteira

Qualquer par que não esteja nesta tabela é recusado com `transicao_invalida`, e nada é gravado. Esta tabela é o teste unitário: cada linha vira um caso, e cada par ausente vira um caso negativo.

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
abrindo	pronto	observando	Grava sessão e catálogo versão 1; abre overlay; inicia o ciclo.
observando	captura_sem_mudanca	observando	Grava ciclo com captura poupada; mantém situação anterior.
observando	situacao_ok	decidindo	Grava situação; monta o pedido de decisão com memória curta.
observando	situacao_fora_de_jogo	bloqueada	Grava situação e, se morte, o marco; congela o contador de intervalo.
observando	situacao_invalida	observando	Grava ciclo sem situação; segunda vez seguida vira *falha_servico*.
decidindo	decisao_nada	observando	Grava decisão "nada" com motivo e origem; overlay discreto.
decidindo	decisao_descartada	observando	Grava decisão com origem descarte e o motivo; trata como "nada".

ESTADO ATUAL	EVENTO	NOVO ESTADO	O QUE ACONTECE JUNTO
decidindo	intervalo_nao_venceu	observando	Grava decisão com origem ritmo; informa ao serviço a próxima janela.
decidindo	decisao_validada	executando	Relê situação e estado; grava estado antes.
executando	situacao_mudou	observando	Grava intervenção descartada por situação velha (RN-08).
executando	execucao_confirmada	observando	Grava estado depois; mostra ação e motivo; marca reversível; zera intervalo.
executando	execucao_falhou	observando	Grava falha da alavanca; conta para tirá-la do catálogo.
bloqueada	cena_voltou	observando	Descongela o contador; grava marco de mudança de fase.
bloqueada	captura_sem_mudanca	bloqueada	Grava ciclo; segue esperando.
observando, decidindo, bloqueada	jogador_pausou	pausada_jogador	Encerra ações com duração; grava marco; overlay "mestre pausado".
executando	jogador_pausou	pausada_jogador	Cancela a execução antes do comando; grava intervenção cancelada pelo jogador.
pausada_jogador	jogador_retomou	observando	Reconstrói memória curta e catálogo; reinicia intervalo; grava marco.
observando, bloqueada, pausada_jogador, silenciada	jogador_reverteu	mesmo estado	Executa a reversão própria da última intervenção; grava; conta como intervenção para o intervalo.
observando, decidindo, bloqueada	falha_servico	silenciada	Grava falha visível; overlay com destaque; nenhuma ação.
qualquer ativo	falha_emulador	silenciada	Grava falha; distingue fechado, travado, canal caído; tenta reconectar.
qualquer ativo	falha_registro	pausada_jogador	Sem como registrar, pausa o mestre e avisa (F01 E1).
silenciada	recuperou	observando	Grava a volta; retoma o ciclo; intervalo tratado como vencido.
silenciada	falha_emulador	encerrada	Só quando o emulador fechou: encerra com resumo marcando a queda (G03).
qualquer, exceto encerrada	jogador_encerrou	encerrada	Encerra ações com duração; estado final em slot reservado; resumo; fecha o arquivo.

Concorrência: quem chega junto. Três coisas acontecem ao mesmo tempo: o serviço demora a responder enquanto o jogador aperta reverter, a captura seguinte chega enquanto a execução espera confirmação, a tecla de pausa chega no meio de uma decisão. A proteção é um só laço: a máquina de estados roda numa fila de eventos única, em ordem de chegada, e cada evento é processado até o fim antes do próximo. Serviço, captura e teclas só enfileiram. Nunca resolvemos isso com trava fina nem com dois laços.

CAPÍTULO 5

Os contratos: o que entra e o que sai de cada porta

Quatro contratos fechados. Dois com o serviço de linguagem, em JSON com lista fechada em todo campo que o mestre decide por cima. Um é o catálogo. O último é a lista de falhas, igual em todo lugar.

Contrato 1 · descrever: o pedido de situação (B02, decisão DC-16)

```
// pedido ao serviço
{
  "tarefa": "descrever",
  "jogo": { "titulo": "Resident Evil 4", "serial":
"SLUS-21134" },
  "estado": { "status": "rodando", "velocidade":
1.0,
              "ultima_alavanca": "recarregar",
"ha_ms": 41200 },
  "situacao_anterior": { "fase": "combate",
"aperto": 3 },
  "captura": "<imagem jpeg, 640 px de largura>",
  "fases_permitidas":
["exploracao","combate","menu","video",

"carregamento","morte","sistema"],
  "escala_aperto": "0 tranquilo … 4 prestes a
morrer"
}
```

```
// resposta aceita (qualquer desvio = inválida)
{
  "descricao": "Leon cercado por quatro aldeões
numa casa, pouca vida",
  "fase": "combate",
  "aperto": 3,
  "confianca": 0.8
}

// regras de validação, na plataforma:
// descricao: 1 linha, até 160 caracteres, em
português
// fase: exatamente uma da lista; aperto: inteiro
0..4
// confianca: 0..1; campo a mais ou a menos =
inválida
// fase 'combate' com status 'pausado' = suspeita
(B02 E3)
```

Contrato 2 · decidir: o pedido de intervenção (C02)

```
// pedido ao serviço
{
  "tarefa": "decidir",
  "tom": "imprevisivel", "intensidade": "media",
  "situacao": { "descricao": "…", "fase":
"combate", "aperto": 3 },
  "estado": { "status": "rodando", "velocidade":
1.0 },
  "catalogo": [
    { "alavanca": "camera_lenta", "disponivel":
true,
      "parametros": { "escalar": [0.5, 0.7],
"duracao_s": [3, 8] } },
    { "alavanca": "recarregar", "disponivel":
true,
      "parametros": { "slot": [93, 94, 95] } },
    { "alavanca": "pausa", "disponivel": false }
  ],
  "memoria": { "ultimas_intervencoes": [ … ],
"ciclos_sem_agir": 6,
               "marcos": [ { "tipo": "morte",
"ha_ms": 95000 } ],
               "revertidas_pelo_jogador":
["turbo"] },
  "proxima_janela_ms": 0
}
```

```
// resposta aceita
{
  "alavanca": "camera_lenta",      // ou "nada"
  "parametros": { "escalar": 0.5, "duracao_s": 5
},
  "motivo": "Quatro contra um e a vida no fim: um
fôlego."
}

// validação dupla, na plataforma (C02):
// 1. alavanca existe e está disponível; cada
parâmetro
//    dentro da faixa; motivo 1 linha até 120
caracteres;
//    regra de mesa: recarregar só para slot com
situação
//    de aperto menor que o atual (RN-02, "não
derrota")
// 2. releitura: status e fase iguais aos do
pedido;
//    senão, descartada por situação velha (RN-08)
// fora disso: descartada inteira; nunca
aproximada
```

Por que o catálogo vai inteiro no pedido. O serviço só pode escolher o que está na lista, com faixas explícitas. Quando uma alavanca fica indisponível, ela continua na lista com `disponivel: false`, para o serviço saber que existe e não existe agora. É mais barato que explicar, e torna o descarte raro: a bancada mede quantas decisões caem na validação, e esse número tem de ser pequeno.

CAPÍTULO 5 · CONTINUAÇÃO

Catálogo, tons, intensidades, teclas e o aviso

Aqui fecham cinco decisões que os casos de uso deixaram como candidatas: DC-15 (slots), DC-17 (tons e intensidades), DC-18 (teclas), DC-19 (aviso) e, junto com o capítulo 6, DC-16 e LC-12.

Contrato 3 · o catálogo universal da fase 1

ALAVANCA	PARÂMETROS E FAIXA	EXECUTA POR	CONFIRMA POR	REVERSÃO	MODO REDUZIDO
`pausa`	duracao_s 1 a 6	A03, comando estendido	status "pausado" e depois "rodando"	retomar na hora	indisponível
`camera_lenta`	escalar 0,5 a 0,8 · duracao_s 3 a 10	A04, comando estendido	consulta de velocidade em vigor	volta ao normal	indisponível
`turbo`	escalar 1,5 a 2,0 · duracao_s 3 a 10	A04, comando estendido	consulta de velocidade em vigor	volta ao normal	indisponível
`salvar_estado`	nenhum; slot escolhido pela fila	A05, canal oficial	arquivo com tamanho estável em duas leituras com 300 ms	não se aplica	disponível
`recarregar`	slot: um dos reservados com contexto, de aperto menor que o atual	A06, canal oficial	status "rodando" e releitura da situação	recarregar o slot de volta	disponível
`nada`	nenhum	C03	registro	não se aplica	disponível

Decisão DC-15, fechada. Slots reservados ao mestre: 90 a 99, fila de 10 por jogo, o mais antigo sobrescrito. Provado no emulador do cliente: o salvamento no slot 90 funcionou e a limpeza de estados do emulador só alcança os slots de 1 a 10. Ações de mundo (fase 2) entram nesta mesma tabela como linhas novas, sem mudar o formato.

Decisão DC-17, fechada · tons e intensidades (C01)

TOM	O QUE MUDA NO PEDIDO DE DECISÃO
Protetor	Prefere trégua: câmera lenta e recarga quando o aperto sobe; turbo só em exploração longa. Motivos acolhedores.
Imprevisível (padrão)	Sem preferência; varia; pode apertar ou aliviar na mesma situação. Motivos curtos e teatrais.
Implacável	Prefere apertar: turbo em combate, recarga só após morte. Nunca cruza a regra "não derrota". Motivos secos.

INTENSIDADE	INTERVALO MÍNIMO	FAIXAS	SALVAR ESTADO A CADA
Leve	60 s	duração até a metade da faixa; sem turbo	90 s
Média (padrão)	30 s	faixa completa	60 s
Forte	15 s	faixa completa; pausa até 6 s	45 s

Decisão DC-18, fechada · os três gestos do jogador (D03, D05, E04)

GESTO	TECLA GLOBAL	BOTÃO NO OVERLAY	POR QUÊ
Reverter a última ação	`Ctrl+Alt+Z`	Desfazer	Combinação que o emulador não usa por padrão e que a mão esquerda alcança sem soltar o controle. Confirmar no esqueleto que não colide com o jogo.
Pausar e retomar o mestre	`Ctrl+Alt+P`	Pausar / Retomar	Idem. Não confunde com Espaço, que pausa o emulador.
Encerrar a sessão	`Ctrl+Alt+Q`	Encerrar	Pede confirmação no overlay; nunca fecha o emulador.

Decisão DC-19, fechada · o aviso do que sai da máquina (E02), versão 1

Para o mestre enxergar o jogo, ele manda capturas da tela para fora. O que sai da sua máquina: imagens reduzidas da área do jogo, uma a cada poucos segundos quando a cena muda, e um resumo em texto do estado da partida (jogo, se está rodando, última ação do mestre). Para onde: o serviço de linguagem configurado, chamado *[nome do adaptador]*. O que nunca sai: o arquivo do jogo, a BIOS, os seus saves, qualquer coisa fora da janela do jogo. Você pode pausar o mestre a qualquer momento com Ctrl+Alt+P, e com a captura desligada nada sai. Este aviso volta a aparecer se o serviço mudar. **[Aceitar e abrir a sessão] [Recusar]**

Contrato 4 · as classes de falha, iguais em todo lugar

ORIGEM	CLASSE	QUANDO ACONTECE	O QUE O MESTRE FAZ
`servico`	`fora_do_ar`	Conexão recusada, tempo esgotado, erro 5xx	Silencia; tenta de novo com intervalo crescente; retoma sozinho.
`servico`	`limite`	Erro de cota ou de taxa	Silencia; espera o intervalo informado ou o padrão; retoma sozinho.
`servico`	`recusa`	O serviço recusou a captura por política própria	Registra com a captura; não insiste com a mesma imagem; segue no ciclo seguinte.
`servico`	`formato`	Resposta fora do contrato pela segunda vez seguida	Silencia até uma resposta válida; conta para o registro de defeitos.
`emulador`	`canal_caido`	Processo e janela vivos, socket fechado	Reconecta (A01); retoma a sessão (E05).
`emulador`	`travado`	Processo vivo, status não responde no limite	Avisa; espera o limite; trata como fechado.
`emulador`	`fechado`	Processo ausente	Encerra a sessão com resumo marcando a queda.
`emulador`	`alavanca_falhou`	Comando aceito, confirmação não veio	Registra; não repete no ciclo; três seguidas tiram a alavanca do catálogo.
`captura`	`janela_ausente`	Janela do emulador não encontrada	Ciclo com lacuna; silencia até a janela voltar.
`registro`	`sem_escrita`	Disco cheio ou pasta sem permissão	Pausa o mestre; avisa; nada executa sem registro.

CAPÍTULO 6

A extensão do canal do emulador

O canal oficial é um socket TCP local com um protocolo binário simples: cada mensagem é um tamanho de 4 bytes, um código de 1 byte e o argumento; a resposta é um tamanho, um resultado (0 ou 255) e o retorno. Vários comandos podem ir no mesmo pacote. A versão própria acrescenta quatro códigos ao servidor desse canal e mais nada.

O QUE O CANAL OFICIAL JÁ TEM (VERIFICADO NO CÓDIGO DA 2.8.2)

- Ler e escrever memória em 8, 16, 32 e 64 bits (códigos 0 a 7).
- Versão do emulador (8); salvar e recarregar estado por slot (9 e 10), assíncronos, com erro só na tela do emulador.
- Título (11), serial (12), soma de verificação (13), versão do jogo (14), status rodando, pausado ou desligado (15).

O QUE A VERSÃO PRÓPRIA ACRESCENTA

- Códigos com o bit mais alto ligado, de 0x80 em diante, para nunca colidir com o que o projeto original vier a criar.
- Pausar e retomar, pelo mesmo ponto de entrada que o atalho do emulador usa.
- Velocidade com modo, escalar e duração, com a volta ao normal dentro do próprio emulador.
- Consulta de capacidades, que é como a plataforma descobre a versão própria (A07).

Os quatro comandos estendidos

CÓDIGO	NOME	ARGUMENTO	RETORNO	O QUE FAZ POR DENTRO
0x80	ExtCapabilities	nenhum	u16 versão da extensão · u32 máscara de bits (1 pausa, 2 velocidade, 4 resultado de estado)	Responde a máscara fixa da versão compilada. No emulador original, o código cai no caso padrão e volta falha: é assim que A07 detecta o modo reduzido.
0x81	ExtSetPaused	u8 pausar (1) ou retomar (0)	nenhum	Executa na thread da CPU a mesma chamada de pausa do emulador. Sem máquina virtual ativa, falha. A confirmação vem pelo status (15).
0x82	ExtSetSpeed	u8 modo (0 normal, 1 lenta, 2 turbo) · f32 escalar · u32 duração em ms	nenhum	Grava o escalar na configuração de velocidade do modo pedido, troca o modo do limitador e atualiza a velocidade alvo. Duração maior que zero arma o vigia: ao vencer, o próprio servidor do canal volta o modo a normal. Modo 0 ou duração 0 desarma o vigia e volta agora.
0x83	ExtGetSpeed	nenhum	u8 modo · f32 velocidade alvo · u32 ms restantes do vigia	Lê o modo do limitador e a velocidade alvo. É a confirmação de A04.

Lacuna LC-12, fechada por desenho. A volta da velocidade ao normal não depende da plataforma: o vigia mora no servidor do canal, dentro do emulador, e dispara pela thread da CPU quando a duração vence. Se o mestre cair com o jogo em câmera lenta, o jogo volta sozinho. O escalar normal do jogador nunca é tocado: a extensão só escreve nos escalares de câmera lenta e turbo e troca o modo do limitador.

Lacuna LC-11, decidida. Salvar e recarregar continuam confirmados pelo arquivo e pela releitura, como as fichas A05 e A06 dizem, porque isso funciona no emulador original e a prova mediu 0,9 s. O bit 4 da máscara de capacidades fica reservado para um código futuro que devolva o resultado do salvamento pelo canal; não é dependência de nada na fase 1.

SEQUÊNCIA: CÂMERA LENTA COM DURAÇÃO, NO EMULADOR, COM A PLATAFORMA CAINDO NO MEIO



O jogo volta ao normal sem a plataforma. O vigia é um prazo verificado no laço do servidor do canal, que já roda numa thread própria; ao vencer, enfileira a volta na thread da CPU, como os comandos de estado já fazem.

Como a versão própria é construída e mantida

ASSUNTO	A REGRA
Repositório	Derivado do PCSX2 na etiqueta corrente do projeto (a única com dependências pré-compiladas publicadas; a 2.8.2 pedia versões anteriores), com commits de extensão isolados por recurso: o do canal toca só `pcsx2/PINE.cpp`; os da camada de cena (fase 4) tocam o renderizador. Repositório público, sob a mesma licença GPL-3.0, com o código da extensão (DC-14). Medido: compilação completa em 7 min, incremental em 3 min (LC-10).
Compilação	A receita do próprio projeto para Windows: MSBuild com a solução Qt, configuração Release, MSVC. As dependências vêm do pacote pré-compilado que o projeto publica (164 MB), extraído em `deps/`; não se compila Qt nem ffmpeg.
Versão	O binário responde à versão do canal com o sufixo da extensão, e à consulta de capacidades com a versão da extensão. A plataforma registra os dois na sessão.
Acompanhar o original	A cada versão estável do projeto, rebase dos commits de extensão e nova compilação. O commit do canal é um arquivo: conflito zero ou trivial. Os da camada de cena tocam a parte que mais muda no original; por isso ficam atrás de uma única função de entrada e de um arquivo próprio, para o rebase ser mecânico (DC-13 revisada).
Proposta ao original	Depois que o esqueleto jogar com a versão própria, o commit vira uma proposta ao projeto original, nos termos de contribuição dele. Aceito, a versão própria deixa de existir.
Ferramentas que faltam na máquina do cliente	Nenhuma obrigatória: MSBuild, MSVC, SDK, CMake e Ninja estão instalados. O extrator de 7z do projeto não está, e o pacote de dependências é extraído pelo Python. Disco não é mais risco: 87 GB livres após limpeza do cliente, e a árvore do projeto com dependências e objetos de compilação fica abaixo de 20 GB. O card S4-01 ainda mede o espaço ocupado, para registro.

A camada de cena (fase 4, DC-20): o que o renderizador vê e o que a camada faz com isso

Todo desenho que o jogo pede chega ao renderizador da versão própria como um lote de triângulos já projetados na tela: para cada vértice, posição em coordenadas de tela com profundidade, coordenada de textura, cor; para o lote, a textura que o veste, o alvo de desenho e o teste de profundidade. O renderizador já identifica a textura por um hash estável entre quadros, que é o que a troca de texturas do próprio emulador usa; esse hash hoje não fica guardado no desenho, e a camada passa a guardá-lo. Não existe câmera nesse ponto: o jogo já multiplicou tudo pela matriz dele antes de enviar. É isso que separa os três níveis.

PEÇA	ONDE MORA	O QUE FAZ
Assinatura de desenho	Renderizador, na função única de entrada da camada, chamada imediatamente antes de cada lote ir para a placa de vídeo	Hash da textura, quantidade de vértices e caixa envolvente na tela. É a identidade de um objeto num quadro (LC-15). A camada mantém a lista das assinaturas do último quadro para o canal ler.
Tabela de regras	Renderizador, protegida por trava; escrita pelo servidor do canal, lida pelo renderizador uma vez por quadro	Cada regra: assinatura alvo, ação (esconder, recolorir, deformar, duplicar com deslocamento; no nível 2, acrescentar), parâmetros e prazo. Prazo vencido apaga a regra sozinho, como o vigia de velocidade. Sem regras, o custo é um teste por lote.
Ações de nível 1	Renderizador	Esconder: o lote não vai para a placa. Recolorir: a cor dos vértices é modulada antes do envio. Deformar: os vértices são escalados em torno do centro da caixa envolvente. Duplicar: o mesmo lote vai duas vezes, o segundo deslocado na tela. Nenhuma depende de câmera; todas respeitam o teste de profundidade que o jogo pediu.
Ações de nível 2	Renderizador + perfil de memória	Acrescentar geometria: a camada recebe pelo canal a matriz de câmera lida da memória do jogo (campo "câmera" do perfil, LC-14), projeta os vértices novos para o espaço de tela do emulador com a mesma convenção de profundidade do jogo (LC-16) e os envia como um lote a mais, com o teste de profundidade ligado, para o que está na frente tapá-los.
Comandos de cena no canal	Servidor do canal, códigos 0x84 em diante	0x84 lista as assinaturas do último quadro; 0x85 grava uma regra (assinatura, ação, parâmetros, prazo); 0x86 apaga regras (uma ou todas); 0x87 entrega a câmera para o nível 2. Falham no emulador original como os demais; a máscara de capacidades ganha o bit 8 (cena).
Perfil de cena	Plataforma, por jogo, versionado como o perfil de memória	Classifica assinaturas conhecidas: cenário, inimigo, item, interface, perigo, saída. É o que permite a RN-11: regra sobre inimigo, perigo ou saída é recusada pela plataforma antes de chegar ao canal. Assinatura sem classe só aceita ações reversíveis e visíveis.
Catálogo	Plataforma	As ações de cena entram como alavancas com faixas, duração e reversão, pelo mesmo caminho das outras: o mestre decide, a plataforma confere contra o perfil de cena e a RN-11, executa pelo canal, confirma lendo as regras ativas, registra antes e depois. Encerrar a sessão apaga todas as regras.

Lacuna LC-15, identidade entre quadros. A mesma textura veste vários objetos (todas as caixas de um corredor), e a caixa envolvente de um objeto muda quando a câmera gira. A prova medida (S7-00) mede quanto a assinatura "hash + vértices + caixa" é estável num objeto parado e num objeto que se move; se não bastar, entra a posição vinda do perfil de memória, que é a resposta definitiva. **Lacuna LC-16, profundidade.** A prova mede, no jogo de abertura, a faixa e o sentido da profundidade que chega ao renderizador, para o nível 2 projetar certo; cada jogo terá o seu valor no perfil de cena.

O perfil de memória (fase 2, DC-22): o que o jogo guarda e como se descobre

A memória principal do PlayStation 2 tem 32 MB e o canal já lê e escreve qualquer endereço dela (códigos 0 a 7). O que falta é saber o que cada endereço significa, e isso é o perfil de memória: por jogo, por versão, versionado, só fatos. A prova de 27/10 mediu o caminho: uma foto da memória inteira pelo canal, em lotes de 50 mil leituras de 8 bytes, leva 3,4 s; os endereços públicos de vida do jogador e da parceira do jogo de abertura foram lidos ao vivo e batem (1200 de 1200 e 600 de 600); e a comparação de quatro fotos numa cena parada já devolveu as matrizes de ossos do personagem animado. Câmera e lista de entidades exigem movimento controlado, que é o card S8-00.

PEÇA	ONDE MORA	O QUE FAZ
Perfil de memória (esquema v1)	Repositório, `perfis/memoria/<serial>.json`, versionado como o perfil de cena	Cabeçalho: serial, soma de verificação, versão do jogo, versão do perfil, data. Variáveis: nome, endereço (ou cadeia de ponteiros), tipo (u8, u16, u32, s32, f32, vec3, mat4), faixa, significado em uma linha, origem (mão, fonte pública com referência, descoberta com data), estado da validação. Compostas: câmera (matriz ou posição mais alvo, com a convenção de eixos) e lista de entidades (endereço do primeiro, passo, quantidade máxima, campos por entidade). Intocável: faixas de endereço proibidas. Ações de mundo: variável, operação, faixa por intensidade, duração, volta, proibições.
Foto da memória	Plataforma, só fora de sessão (bancada e ferramenta do mapeador)	Pausa pelo canal (0x81), 32 MB em lotes de 50 mil leituras de 8 bytes, retoma. Sem a pausa a foto mistura instantes e a comparação mente. Dentro de uma sessão nunca se fotografa: lê-se variável por variável (A08).
Comparação de fotos	Plataforma	Entre fotos com movimento: números de ponto flutuante finitos, pequenos, que mudam aos poucos; entre fotos sem movimento: os que ficam iguais. Agrupamento por vizinhança revela vetores (3 ou 4 floats) e matrizes (linhas unitárias, 1.0 no fim). Passo constante entre blocos iguais revela a lista de entidades e o tamanho de cada uma. Busca por valor conhecido para o que aparece na tela.
Separar câmera de jogador	Plataforma + mapeador	Dois movimentos: andar sem virar e virar sem andar. O que muda nos dois é o jogador (e a câmera que o segue); o que muda só ao virar é a câmera. Um inimigo andando sozinho dá a entidade que não é o jogador.
Validação por projeção	Plataforma, com a camada de cena	Posição da entidade projetada pela câmera candidata tem de cair dentro da caixa do que a camada de cena vê se mover, em quase todos os quadros amostrados. É o critério que fecha câmera e entidades juntas, e o que a RN-11 e a cena de nível 2 passam a usar (LC-14, LC-15).
Cadeia de ponteiros	Perfil	Endereço que muda entre áreas ou partidas é gravado como cadeia: endereço fixo que guarda um endereço, e assim até o alvo. A plataforma resolve a cadeia a cada leitura e marca a variável como suspeita se um elo sair da memória.
Ações de mundo	Catálogo, por jogo	Entram pelo mesmo caminho das universais: faixa por intensidade, duração, volta declarada, proibições do mapeador mais as regras de mesa; a execução lê antes, escreve, relê e registra (A08, H07). Nenhuma ação sem volta, nenhuma que possa derrotar sozinha.

Três níveis de engenharia reversa, um por vez. Este capítulo cobre o nível 1 (memória). O nível 2 (código: funções do jogo, pelo depurador do emulador) entra quando uma ação de mundo precisar de algo que a memória não dá, como criar um inimigo. O nível 3 (formatos de modelos, colisão e fases) é a lacuna LC-17 e será fase própria.

CAPÍTULO 7

Três fluxos ponta a ponta

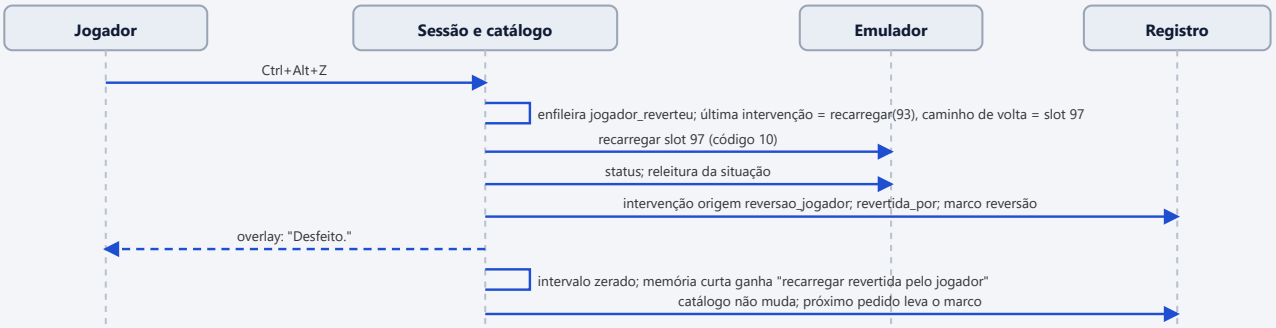
Os diagramas abaixo amarram o que veio até aqui: a captura, o pedido ao serviço, a validação, o comando ao emulador e o registro, na ordem real em que acontecem.

FLUXO 1 · UM CICLO COM INTERVENÇÃO



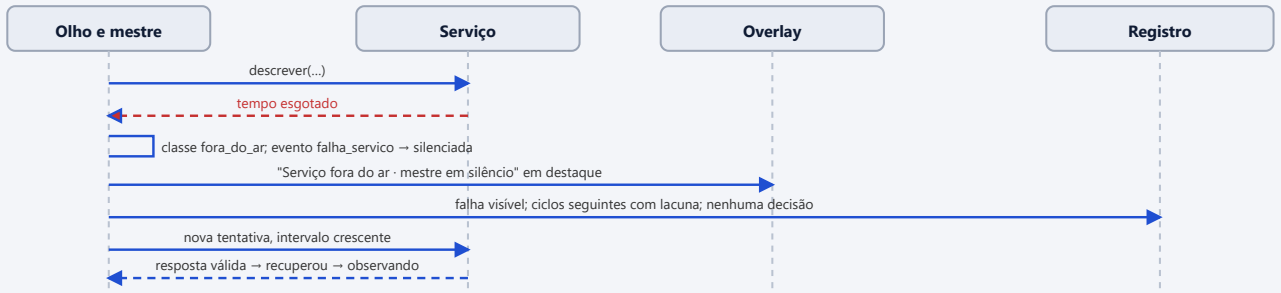
O serviço aparece duas vezes e nunca toca o emulador. A plataforma grava antes de agir e depois de confirmar. Em modo reduzido, o mesmo fluxo com recarregar: o comando é o código 10 e a confirmação é o status mais a releitura.

FLUXO 2 · O JOGADOR REVERTE UMA RECARGA



A reversão de uma recarga é a única recarga que não salva antes: o estado atual é o slot 97, guardado pela própria recarga que está sendo desfeita.

FLUXO 3 · O SERVIÇO CAI NO MEIO DA SESSÃO



O jogo nunca para. O que estava em câmera lenta volta pelo vigia do emulador; o que não foi decidido não é decidido. É a regra RN-10 em desenho.

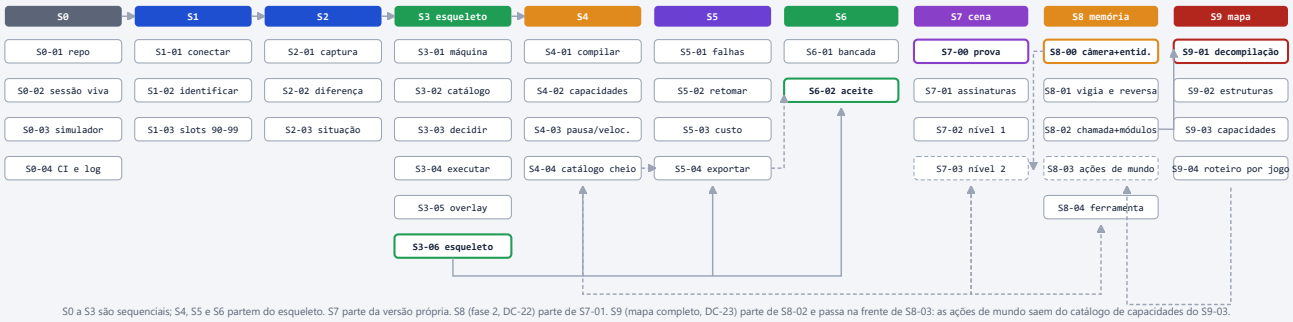
CAPÍTULO 8

O roadmap em 35 cards

Daqui para frente o documento vira backlog. Cada card tem um identificador permanente, um motivo de existir, a lista do que entra no repositório e um critério de aceite escrito como teste. A ordem é de execução, não de importância. Não há estimativa de esforço: o que mede o avanço é card fechado com critério provado.

ETAPA	O QUE MUDA NO FIM DELA	CARDS
S0 Fundação	Repositório clonável, sessão viva criada e provada, simulador do canal respondendo, integração contínua verde. Ninguém escreve regra ainda.	4
S1 Ponte	O mestre conecta, identifica o jogo, salva e recarrega nos slots reservados com confirmação, contra o emulador real e contra o simulador.	3
S2 Olho	Captura pela janela, só o que mudou vai ao serviço, situação em formato fechado validada.	3
S3 Mestre e esqueleto	Máquina de estados, decisão com validação dupla, catálogo reduzido, overlay com três gestos, registro completo. **É a primeira coisa que o cliente joga.**	6
S4 Versão própria do emulador	Compilar o original sem mudar nada (a medição da lacuna LC-10), depois os quatro comandos, depois pausa e velocidade no catálogo.	4
S5 Robustez	Falhas classificadas, retomada, custo com teto, exportação e reprodução.	4
S6 Bancada e sessão jogada	Bancada no CI com casos sintéticos, comparação de serviços, e a sessão de aceite da fase 1.	2
S7 Camada de cena	A prova medida que decide a fase 4 (DC-21), depois assinaturas pelo canal, as ações de nível 1 no catálogo com perfil de cena e RN-11, e o nível 2 quando a fase 2 entregar a câmera.	4
S8 Perfil de memória	A prova medida com o cliente no controle (câmera e entidades do jogo de abertura, validadas por projeção), o perfil de memória v1 com leitura e escrita pelo canal, a validação automática, as ações de mundo no catálogo e a ferramenta do mapeador.	5

QUEM DEPENDE DE QUEM



S0 a S3 são sequenciais; S4, S5 e S6 partem do esqueleto. S7 parte da versão própria. S8 (fase 2, DC-22) parte de S7-01. S9 (mapa completo, DC-23) parte de S8-02 e passa na frente de S8-03: as ações de mundo saem do catálogo de capacidades do S9-03.

Como usar estes cards. Cada card vira um item no quadro, com o identificador no título. O critério de aceite é copiado como está: é a definição de terminado e deve ser executado por outra pessoa, não pelo autor; no caso deste projeto, pelo cliente, que também é o jogador. Trabalho novo entra como card novo, com código próprio.

ETAPA S0

Fundação

4 cards

Ninguém escreve regra de negócio ainda. O objetivo é repositório, sessão viva, simulador e integração contínua, para que todo card seguinte comece com o chão pronto.

S0-01 Repositório, pacotes e configuração

Deixar o projeto clonável e rodando em um comando, com a estrutura de pacotes do capítulo 2 já no lugar.

- Pasta `mestre/` com os nove pacotes vazios e um ponto de entrada `python -m mestre`.
- Dependências fixadas: captura e janelas, pillow, pydantic, httpx, pytest, ruff.
- Configuração em arquivo TOML: porta do canal, pasta de sessões, slots reservados, resolução da captura, limiar de diferença, serviço ativo, teto de custo; chave do serviço só por variável de ambiente, nunca no repositório.
- README com o fluxo: abrir o emulador, abrir o mestre, teclas.

CRITÉRIO DE ACEITE

Dado um clone limpo, **quando** se instala e roda o ponto de entrada sem emulador, **então** ele sobe, informa "canal não encontrado" e encerra limpo, sem traceback.

S0-02 Sessão viva: esquema, gatilhos e provas

UC-F01 UC-F02

Ter o registro provado antes de qualquer coisa escrever nele: intervenção sem registro não existe.

- Migração 0001 do capítulo 3, aplicada ao criar o arquivo da sessão.
- Gatilhos de só acréscimo com as exceções nomeadas de nulo para valor.
- Camada de acesso: abrir sessão, gravar ciclo, captura, situação, decisão, intervenção, falha, métrica, marco; nenhuma função de alterar ou apagar existe.
- Teste com as três tentativas proibidas: alterar ciclo fechado, apagar decisão, gravar fase fora da lista.

CRITÉRIO DE ACEITE

Dado um arquivo de sessão novo, **quando** os três comandos proibidos rodam, **então** os três falham com "registro só de acréscimo" ou violação de CHECK e nada muda; e uma cadeia ciclo, captura, situação, decisão, intervenção grava e é lida de volta inteira.

S0-03 Simulador do canal do emulador

UC-A01 UC-A07

Testar a ponte e o ciclo sem emulador, no notebook e no CI, inclusive os comandos estendidos antes de a versão própria existir.

- Servidor TCP em Python que fala o protocolo binário: tamanho, código, argumento; resposta com resultado 0 ou 255.
- Jogo de mentira: título, serial, soma, status, memória de 32 MB zerada, slots que gravam arquivo na pasta configurada com atraso simulado de 0,9 s.
- Modo "original" (só códigos 0 a 15) e modo "próprio" (mais 0x80 a 0x83 com vigia de velocidade).
- Cenários de falha: canal cai, status trava, salvamento nunca aparece, jogo troca de serial no meio.

CRITÉRIO DE ACEITE

Dado o simulador em modo original, **quando** a ponte pede capacidades, **então** recebe falha e marca modo reduzido; em modo próprio, recebe a máscara e marca modo completo.

S0-04 Integração contínua e log estruturado

Que todo push prove o que não depende de emulador nem de serviço, e que qualquer falha deixe rastro.

- Fluxo no GitHub: lint, tipos, testes unitários e os cenários do simulador, a cada push.
- Log em JSON com identificador da sessão e número do ciclo em toda linha.
- O CI nunca tem chave de serviço; testes que precisam de serviço são marcados e pulados nele.

CRITÉRIO DE ACEITE

Dado um push que quebra um cenário do simulador, **quando** o CI roda, **então** o build falha e aponta o cenário; e nenhuma variável de serviço existe no ambiente do CI.

ETAPA S1

Ponte

3 cards

O mestre passa a conversar com o emulador. No fim desta etapa você salva e recarrega um estado reservado pelo terminal, contra o emulador real e contra o simulador, com a mesma chamada.

S1-01 Cliente do canal: conectar e sondar

UC-A01

Ter um canal aberto e confirmado antes de qualquer outra coisa, tolerando o carregamento do jogo.

- Cliente do protocolo: conectar, versão, status, com tempo limite por chamada e reconexão.
- Sondagem até "rodando" com limite configurado; tratamento do "desligado" e do erro 255 nos segundos após o lançamento.
- Detecção de porta fechada e de outro programa na porta pela resposta à versão.

CRITÉRIO DE ACEITE

Dado o emulador lançado com o jogo, **quando** a ponte conecta no mesmo instante, **então** ela espera, e marca "canal pronto" só quando o status vira "rodando", registrando a versão do emulador.

S1-02 Identificar o jogo e o modo

UC-A02 UC-A07

Nunca agir sobre o jogo errado e saber se o emulador é a versão própria.

- Título, serial, soma, versão do jogo pelo canal; registro na sessão.
- Consulta de capacidades (0x80): falha significa modo reduzido.
- Detecção de troca de jogo entre sondagens e de serial vazio.

CRITÉRIO DE ACEITE

Dado o jogo de abertura rodando, **quando** a ponte identifica, **então** devolve serial SLUS-21134 e soma 013e349d, e o modo é "reduzido" no emulador original.

S1-03 Slots reservados com confirmação pelo arquivo

UC-A05 UC-A06

Salvar e recarregar estado do mestre sem nunca tocar nos slots do jogador, com confirmação real.

- Fila de slots 90 a 99 por jogo; escolha do mais antigo; registro em estados_reservados.
- Confirmação: arquivo com o nome calculado existe e tem tamanho igual em duas leituras com 300 ms; limite de 10 s.
- Recarga: salvar antes em slot reservado, recarregar, confirmar por status; caminho de volta registrado.
- Recusa de configuração em que um slot reservado esteja entre 1 e 10.

CRITÉRIO DE ACEITE

Dado o jogo rodando, **quando** a ponte salva no slot da fila e recarrega, **então** o arquivo é confirmado em menos de 3 s, o status volta a "rodando", e os slots 1 a 10 continuam intocados.

ETAPA S2

Olho

3 cards

O mestre passa a ver. No fim desta etapa, uma captura do jogo vira uma linha de situação validada, e só vai ao serviço se mudou.

S2-01 Captura pela janela com recorte

UC-B01

Pegar o jogo e só o jogo, mesmo com outra janela por cima, como a prova mostrou.

- Localizar a janela pelo processo do emulador, não pelo título.
- Captura pela janela com conteúdo completo, deslocada para a área do cliente: a prova trouxe a barra de menu por não deslocar.
- Recorte da área do overlay quando ele estiver dentro da janela; redução para 640 px; JPEG com qualidade 80.
- Lacuna registrada quando a janela não existe ou está minimizada; teste de tela cheia sem borda (lacuna LC-13).

CRITÉRIO DE ACEITE

Dado o jogo com outra janela por cima, **quando** o olho captura, **então** a imagem mostra o jogo sem barra de menu e sem o overlay, com até 80 KB, em menos de 100 ms.

S2-02 Diferença entre capturas e tempo máximo

UC-B03

Mandar ao serviço só o que mudou, e mesmo assim mandar de tempos em tempos.

- Medida de diferença sobre a imagem reduzida em cinza; limiar configurado; referência trocada a cada envio.
- Tempo máximo sem envio configurado por intensidade.
- Contagem de enviadas e poupadas por sessão em metricas.

CRITÉRIO DE ACEITE

Dado o jogo parado numa tela, **quando** passam dez ciclos, **então** no máximo um envio acontece pelo tempo máximo e os outros são registrados como poupados.

S2-03 Descrever: adaptador do serviço e formato fechado

UC-B02 UC-B04

Transformar imagem em situação que o mestre consegue usar, e recusar o que não cabe no contrato.

- Interface do serviço com descrever e decidir; primeiro adaptador para o provedor cujo acesso o cliente já tem; respostas gravadas para teste.
- Contrato 1 com validação por pydantic: campo a mais ou a menos é inválido.
- Suspeita quando a fase contradiz o status; bloqueio por fase fora de jogo; marco de morte.
- Latência e tokens em metricas; resposta bruta em situacoes.

CRITÉRIO DE ACEITE

Dado uma resposta gravada com fase fora da lista, **quando** o olho valida, **então** a situação é inválida, o ciclo fica sem situação e nada é inventado; e uma captura real do jogo de abertura devolve fase e aperto válidos.

ETAPA S3

Mestre e esqueleto

6 cards

Nasce o mestre. No fim desta etapa o cliente joga uma sessão inteira em modo reduzido: o mestre vê, decide, salva e recarrega, explica, e o jogador reverte e pausa.

S3-01 Máquina de estados da sessão

UC-D04 UC-D05

Centralizar num lugar só toda a regra de quando a sessão muda de estado.

- Função pura transicao(estado, evento) com a tabela do capítulo 4 inteira.
- Fila única de eventos; serviço, captura e teclas só enfileiram.
- Contador monotônico da sessão; intervalo mínimo congelado em bloqueada.
- Testes cobrindo os oito estados e todas as transições, inclusive as inválidas.

CRITÉRIO DE ACEITE

Dado o estado executando, **quando** chega situacao_ok, **então** a transição é recusada com transicao_invalida e nada é gravado; e todo par da tabela tem teste verde.

S3-02 Catálogo, tons e intensidades

UC-C01 UC-D01

A lista fechada do que o mestre pode fazer, com faixas e disponibilidade, versionada.

- Catálogo do capítulo 5 com disponibilidade por modo e por falhas repetidas; motivo padrão por alavanca.
- Tons e intensidades da decisão DC-17 aplicados às faixas e ao intervalo.
- Nova versão do catálogo gravada a cada mudança; tela de escolha antes da sessão; recusa de pedido de ação durante.

CRITÉRIO DE ACEITE

Dado o modo reduzido e intensidade leve, **quando** o catálogo é montado, **então** pausa, câmera lenta e turbo estão indisponíveis, o intervalo é 60 s e a versão 1 está gravada.

S3-03 Decidir: pedido, validação dupla, "nada" e motivo

UC-C02 UC-C03 UC-C04 UC-C05

O serviço escolhe; a plataforma confere duas vezes; o silêncio é decisão.

- Contrato 2 com memória curta derivada do registro (últimas intervenções, marcos, ciclos sem agir, revertidas).
- Validação 1: catálogo, faixa, regra de mesa (recarregar só para aperto menor). Validação 2: releitura de status e fase.
- Descarte inteiro, nunca aproximação; "nada" gravado com motivo e origem; motivo conferido e substituído pelo padrão quando impróprio.
- Testes com respostas gravadas para cada caminho de descarte.

CRITÉRIO DE ACEITE

Dado uma resposta gravada pedindo recarregar um slot de aperto maior que o atual, **quando** o mestre valida, **então** a decisão é descartada por regra de mesa, gravada com origem descarte, e o ciclo vale "nada".

S3-04 Executar e reverter pelo caso de uso de cada alavanca

UC-D02 UC-D03

Nenhuma ação chega ao emulador sem passar pelo catálogo, pelo intervalo e pelo registro, e toda ação tem volta.

- Execução despachada por alavanca para a ponte; estado antes e depois em intervencoes.
- Reversão própria por alavanca; a última intervenção reversível; reversão conta para o intervalo.
- Falha de execução gravada; três seguidas tiram a alavanca do catálogo com nova versão.

CRITÉRIO DE ACEITE

Dado uma recarga executada, **quando** o jogador reverte, **então** o slot de volta é recarregado, a intervenção fica revertida_por e o próximo pedido ao serviço leva o marco de reversão.

S3-05 Overlay, aviso e teclas

UC-E01 UC-E02 UC-E03 UC-E04

Que o jogador veja e controle o mestre sem tirar as mãos do jogo.

- Painel Tk sempre por cima, arrastável, fora da área capturada, com situação, ação, motivo, indicador de envio e tempo até a próxima janela.
- Aviso da decisão DC-19 com aceite gravado em aceites; reapresentado ao trocar de serviço.
- Teclas globais da decisão DC-18; confirmação para encerrar; resumo montado do registro.

CRITÉRIO DE ACEITE

Dado uma intervenção confirmada, **quando** ela acontece, **então** ação e motivo aparecem no painel no mesmo segundo; e Ctrl+Alt+P pausa o mestre sem pausar o jogo.

S3-06 Esqueleto ambulante: primeira sessão jogada

21 casos em modo reduzido

Provar a arquitetura de ponta a ponta antes de qualquer funcionalidade ganhar corpo. É o primeiro item que o cliente joga.

- Tudo de S0 a S3 integrado no ponto de entrada; configuração padrão para o jogo de abertura.
- Medição do ciclo: latência de cada etapa e custo por hora (lacuna LC-06) gravados e mostrados no resumo.
- Registro de defeitos aberto: cada intervenção estranha vira item com causa.

CRITÉRIO DE ACEITE

Dado o emulador original com o jogo de abertura, **quando** o cliente joga uma sessão até encerrar, **então** toda intervenção está no registro com antes, depois e motivo, nenhuma fora do catálogo, e o resumo traz custo e latência.

ETAPA S4

Versão própria do emulador

4 cards

Primeiro compilar o original sem mudar nada, que é a medição da lacuna LC-10. Só depois os quatro comandos, e só depois pausa e velocidade no catálogo.

S4-01 Compilar o emulador original, sem mudar nada

Fechar a lacuna LC-10 com número: espaço em disco, ferramentas que faltam, e se o binário resultante roda o jogo e responde ao canal.

- Repositório derivado na etiqueta v2.8.2, com submódulos; dependências pré-compiladas do projeto extraídas em deps/.
- Compilação pela receita do projeto: MSBuild, solução Qt, Release, MSVC.
- Medição gravada no repositório: espaço ocupado, tempo de máquina, o que faltou instalar.
- Binário próprio roda o jogo de abertura e responde ao canal com a mesma identificação.

CRITÉRIO DE ACEITE

Dado a máquina do cliente, **quando** a receita roda, **então** existe um pcsx2-qt.exe próprio que abre o jogo de abertura e responde ao serial pelo canal; e a medição de disco está gravada. Se o disco não bastar, o card termina com esse número e a decisão de onde compilar.

S4-02 Capacidades: o primeiro comando estendido

UC-A07

O menor diff possível que prova o caminho inteiro: código novo no servidor do canal, resposta, plataforma detectando.

- Código 0x80 no enum e no despacho de PINE.cpp, avançando o contador do buffer; máscara e versão da extensão.
- Sufixo da extensão na resposta à versão.
- Teste de regressão: todos os 16 códigos originais seguem iguais.

CRITÉRIO DE ACEITE

Dado o binário próprio, **quando** a ponte pede capacidades, **então** recebe versão 1 e máscara 3; e no binário original recebe falha, como antes.

S4-03 Pausa e velocidade com vigia

UC-A03 UC-A04

As duas alavancas que faltam, com a volta ao normal dentro do emulador.

- 0x81 pausa pela chamada de pausa do emulador na thread da CPU; 0x82 velocidade pelos escalares de lenta e turbo, troca do modo do limitador e atualização do alvo; 0x83 consulta.
- Vigia no laço do servidor do canal: prazo por comando, volta a normal ao vencer, cancelado por modo 0 ou duração 0.
- Simulador atualizado para o mesmo comportamento, inclusive o vigia.

CRITÉRIO DE ACEITE

Dado câmera lenta de 5 s pedida pelo canal, **quando** o cliente do canal é morto no segundo 2, **então** o jogo volta à velocidade normal no segundo 5 sozinho, e a consulta de velocidade confirma.

S4-04 Catálogo completo e proposta ao projeto original

UC-D01

Pausa e velocidade entram no catálogo pelo mesmo caminho das outras, e o commit vira proposta.

- Alavancas pausa, camera_lenta e turbo executadas e revertidas pela ponte; disponibilidade pela máscara.
- Sessão jogada em modo completo pelo cliente.
- Proposta de contribuição ao projeto original com o commit de extensão, nos termos dele.

CRITÉRIO DE ACEITE

Dado o binário próprio e o jogo de abertura, **quando** o mestre decide câmera lenta, **então** a intervenção é confirmada pela consulta de velocidade e volta ao normal no fim da duração, registrada com início e fim.

ETAPA S5 **Robustez** 4 cards

O mestre passa a falhar bem: visível, em silêncio, retomando sozinho, com custo sob teto e registro exportável.

S5-01 Falhas classificadas do serviço e do emulador

UC-G02 UC-G03

Nunca parecer vivo estando morto, e nunca deixar o jogo alterado.

- Contrato 4 inteiro: classes, destaque no overlay, tentativas com intervalo crescente, pausa automática após o limite.
- Distinção fechado, travado, canal caído pelo processo, pela janela e pelo socket.
- Ao reconectar, conferir velocidade e status e devolver ao normal se preciso.
- Cenários no simulador para cada classe.

CRITÉRIO DE ACEITE

Dado o serviço respondendo erro 503 por três ciclos, **quando** a sessão roda, **então** o overlay mostra a falha em destaque, nenhuma ação acontece, e no quarto ciclo com resposta válida o mestre retoma sozinho, tudo registrado.

S5-02 Retomar sessão interrompida

UC-E05

Nada se perde numa queda, e o mestre nunca retoma sobre um jogo que mudou.

- Detecção de sessão sem fim; oferta de retomar ou abrir nova; retomada só com o mesmo serial e soma.
- Reconstrução de memória curta e catálogo a partir do registro; lacuna com duração; intervalo como vencido.
- Registro corrompido preservado e marcado como irrecuperável.

CRITÉRIO DE ACEITE

Dado o mestre morto no meio de uma sessão, **quando** ele reabre com o mesmo jogo, **então** retoma a mesma sessão com a lacuna gravada e a memória curta igual à de antes da queda.

S5-03 Custo e latência com teto

UC-G01

Decidir frequência, resolução e serviço com número, e nunca estourar o bolso do jogador.

- Tabela de preço por adaptador; custo por envio pelo tamanho; acumulado por sessão e por hora de jogo.
- Teto configurado que pausa o mestre ao ser atingido; aviso ao passar de limites.
- Resumo da sessão com mediana e máximo de latência de decisão e capturas poupadas.

CRITÉRIO DE ACEITE

Dado um teto de sessão configurado, **quando** o custo acumulado o atinge, **então** o mestre pausa, o overlay diz por quê, e a retomada só acontece pelo jogador.

S5-04 Exportar e reproduzir

UC-F03 UC-F04

Entender uma sessão sem o jogo, e levar o registro para fora sem levar o que não é do jogador.

- Exportação em resumo ou completa, com ou sem capturas, com aviso sobre as imagens.
- Reprodução pela linha do tempo: ciclos, decisões, descartes, reversões, marcos; escolha de ciclos para a bancada.
- Leitura tolerante de registro de versão anterior.

CRITÉRIO DE ACEITE

Dado uma sessão jogada, **quando** o mapeador a reproduz, **então** vê cada decisão como veio do serviço, o resultado da validação e o que foi executado, e exporta três ciclos como casos da bancada.

ETAPA S6

Bancada e sessão de aceite

2 cards

Provar o mestre sem jogo e fechar a fase 1 com a sessão jogada pelo cliente.

S6-01 Bancada de situações sintéticas

UC-G04

A única forma repetível de medir o mestre e de escolher o serviço com número.

- Banco de casos versionado: capturas sintéticas, estado, catálogo, tom, regra testada, decisões aceitáveis; capturas reais só em casos locais.
- Rodada só da plataforma com respostas gravadas, no CI; rodada com serviço sob demanda, com custo informado; dois adaptadores lado a lado.
- Relatório com aprovação por regra, consistência entre repetições, latência e custo, guardado com data e versão do pedido.

CRITÉRIO DE ACEITE

Dado o banco de casos, **quando** a rodada roda contra dois adaptadores, **então** sai um relatório comparável com a taxa de decisões descartadas pela validação por adaptador, e o CI roda a rodada gravada sem chave de serviço.

S6-02 Sessão de aceite da fase 1

UC-A01 a UC-G04

O portão da fase: o cliente joga do início ao fim com toda intervenção registrada e nenhuma fora do catálogo.

- Sessão inteira no jogo de abertura, em modo completo, com a intensidade padrão.
- Registro de defeitos revisado; lacunas republicadas; revisão pós-fase escrita.
- Validação da lacuna LC-08 planejada antes de qualquer trabalho da fase 2.

CRITÉRIO DE ACEITE

Dado uma sessão jogada até encerrar, **quando** o registro é reproduzido, **então** toda intervenção tem antes, depois e motivo, nenhuma decisão executada está fora do catálogo, e o cliente diz que o jogo deixou de ser o de sempre, ou diz por que não, e isso vira item.

ETAPA S7

Camada de cena (fase 4)

4 cards

Primeiro a prova medida, que é a decisão DC-21: se a camada não for viável no jogo de abertura, a etapa termina no S7-00 com os números. Depois, só sobre a versão própria, as assinaturas pelo canal, o nível 1 no catálogo e, quando a fase 2 entregar a câmera, o nível 2.

S7-00 Prova medida: esconder um objeto e desenhar outro com profundidade certa

UC-K01 UC-K02

O menor experimento que responde às três lacunas da fase 4 com número, antes de qualquer desenho de produto.

- Função única de entrada da camada no renderizador, chamada antes de cada lote ir para a placa; assinatura (hash da textura, vértices, caixa) gravada num arquivo por quadro durante a prova.
- Regra fixa de teste: esconder a assinatura de um objeto escolhido do jogo de abertura; medir em quantos quadros seguidos ele some sem piscar, parado e com a câmera girando (LC-15).
- Lote injetado: um retângulo em espaço de tela numa profundidade escolhida; medir se é tapado pelo que está na frente e se tapa o que está atrás, e a faixa e o sentido da profundidade do jogo (LC-16).
- Custo por quadro com a camada ligada e sem regras, com uma regra e com dez, medido no renderizador.

CRITÉRIO DE ACEITE

Dado o jogo de abertura na versão própria, **quando** a prova roda, **então** existe um registro com: o objeto escondido em pelo menos 95% dos quadros de 10 segundos parado e o número com a câmera girando; o retângulo injetado ocluído corretamente em três profundidades; e o custo por quadro em milissegundos nos três cenários. Com esses números o cliente decide se S7-01 a S7-03 entram.

S7-01 Assinaturas pelo canal e perfil de cena mínimo

UC-K01 UC-K03

A plataforma passa a ver a cena como lista de objetos identificáveis, e a classificá-los por jogo.

- Códigos 0x84 (lista do último quadro) e 0x86 (apagar regras) no servidor do canal; bit 8 na máscara de capacidades.
- Perfil de cena do jogo de abertura: arquivo versionado com assinaturas conhecidas e classes (cenário, inimigo, item, interface, perigo, saída), criado pelo mapeador com uma ferramenta que mostra a captura e as assinaturas lado a lado.
- Simulador do canal em modo próprio respondendo aos códigos novos, com uma cena de mentira.

CRITÉRIO DE ACEITE

Dado o jogo de abertura rodando, **quando** a ponte pede a lista, **então** recebe as assinaturas do último quadro em menos de 5 ms, e o perfil de cena classifica pelo menos as assinaturas de inimigo e de interface da primeira área do jogo.

S7-02 Ações de nível 1 no catálogo, com RN-11 e reversão

UC-K02 UC-K06

Esconder, recolorir, deformar e duplicar viram alavancas do mestre pelo mesmo caminho das outras.

- Código 0x85 (gravar regra com prazo) e a tabela de regras no renderizador, com trava e vigia de prazo.
- Alavancas de cena no catálogo, com faixas por intensidade; a plataforma recusa regra sobre assinatura classificada como inimigo, perigo ou saída (RN-11) e sobre assinatura sem classe quando a ação não for reversível.
- Reversão pelo jogador e limpeza ao encerrar; confirmação pela leitura das regras ativas; registro antes e depois.

CRITÉRIO DE ACEITE

Dado uma situação de exploração com uma caixa classificada como cenário, **quando** o mestre decide escondê-la por 10 s, **então** ela some, a intervenção é confirmada pela regra ativa e volta sozinha no prazo; e uma decisão de esconder o inimigo é descartada antes de chegar ao canal, registrada com o motivo RN-11.

S7-03

Nível 2: geometria no mundo com a câmera do perfil de memória

UC-K04

A cena ganha o que não existia, com profundidade certa. Depende da fase 2 entregar o campo "câmera".

- Código 0x87 (câmera do quadro) e a projeção dos vértices novos para o espaço de tela do emulador com a convenção de profundidade medida no S7-00.
- Ação "acrescentar" no catálogo, com formas simples (bloco, placa, luz falsa por vértice colorido) e prazo.
- Prova no jogo de abertura: um bloco posto no corredor é tapado pela parede quando a câmera passa atrás dela.

CRITÉRIO DE ACEITE

Dado o perfil de memória com a câmera validada, **quando** o mestre acrescenta um bloco a 2 m do jogador, **então** o bloco fica parado no mundo enquanto a câmera gira, é ocluído corretamente, e some no prazo; tudo registrado.

ETAPA S8

Perfil de memória (fase 2, adiantada)

5 cards

A engenharia reversa do jogo no nível 1, memória. Começa pela prova medida com o cliente no controle, porque câmera e entidades só aparecem com movimento; a validação por projeção na tela é o portão de cada perfil.

S8-00 Prova medida: câmera e lista de entidades do jogo de abertura

UC-H03 UC-I01

Fechar LC-14 e a metade que falta de LC-15 com número, e provar o método de descoberta antes do perfil existir.

- Fotos da memória pausada pelo canal em cada parada de um roteiro de um minuto com o cliente: andar sem virar, parar; virar sem andar, parar; inimigo andando sozinho.
- Comparação das fotos: candidatos a jogador, câmera (matriz com linhas unitárias) e lista de entidades (passo constante); vida do jogador como variável de controle, já conferida.
- Projeção de cada entidade candidata pela câmera candidata contra as caixas da camada de cena, por alguns segundos.
- Medição gravada no repositório, sem imagem nem foto da memória.

CRITÉRIO DE ACEITE

Dado o roteiro de movimentos feito pelo cliente, **quando** a comparação roda, **então** existe um par câmera e lista de entidades cuja projeção cai na caixa do inimigo que se move em pelo menos 90% dos quadros amostrados, com o passo entre entidades e os campos de posição anotados; ou o registro diz quais candidatos sobraram e por que não bastou.

S8-01 Vigia de memória e engenharia reversa do código

UC-H01 UC-H02 UC-A08 feito em 28/10

O que o card previa (perfil v1 e acesso por nome) ficou menor que o que a medição exigiu: a vida do inimigo e a munição não apareciam por diferença de fotos, e o cliente decidiu destrinchar o código do jogo.

- Vigia de memória na versão própria do emulador (códigos 0x89 a 0x8B): por instrução do jogo, cada acesso a uma faixa, sem pausar; achou a função de dano, a vida do inimigo (+0x4B0 da entidade) e a dificuldade adaptativa.
- Executável com os próprios símbolos (3448 nomes, 1275 endereços); projeto do Ghidra sem interface, completado com a foto da memória; roteiro versionado em `reversa/`.
- Munição fechada pelo gerenciador de itens; maleta lida como tabuleiro de peças; item dado pelo canal (lança-foguetes) na abertura seguinte.
- Perfil v1 do jogo de abertura com variáveis, câmera, entidades, inventário e funções; medições em `docs/medicoes/S8-01-vigia.md`.

CRITÉRIO DE ACEITE (COMO FICOU)

Dado o jogo de abertura na versão própria, **quando** o cliente atira e abre a maleta, **então** o vigia aponta a instrução que escreve a vida do inimigo e o item gravado pelo canal aparece na maleta. Cumprido em 28/10, com o cliente no controle.

S8-02 Chamada remota, personagens e módulos carregados pelo mestre

UC-H05 UC-I01 · feito em 29/10

"Spawna um mob": criar personagem é uma função do jogo, não um endereço. A versão própria passou a executar funções do jogo; o limite de quatro módulos de personagem carregados foi contornado pelo mestre carregando módulos da ISO por conta própria.

- Chamada remota (código 0x8C): registradores salvos, argumentos, pilha própria, trampolim, restauração; imediata no vsync ou ancorada num acesso de memória para rodar na thread do jogo. Regra: função que dorme pelo escalonador do jogo trava a thread.
- Provado ao vivo: Ganados, corvos e outros tipos criados onde pedido; itens pela lógica "cabe na maleta"; dinheiro; vida; teleporte; cenário escondido pela camada de cena.
- Módulos: leitor do AFS da ISO, formato SNR2 lido no ligador do jogo, carga pela memória com alocação pelo próprio jogo; identidade dos tipos lida nas tabelas de seções (Ashley, Luis, lanchas, Ada jogável e de cena, Hunk, Krauser, Wesker).
- Fica em aberto a Ada de cena (trava na colisão segundos depois) e o Leon de cena (ligação não termina); medições em `docs/medicoes/S8-02-chamada-remota.md`.

CRITÉRIO DE ACEITE (COMO FICOU)

Dado o jogo de abertura na versão própria, **quando** o mestre pede um aldeão a dois metros do jogador, **então** ele aparece e anda na direção do jogador em menos de um segundo, e o jogo segue; com um tipo ausente, o módulo é carregado da ISO e o personagem aparece sem tela de loading. Cumprido em 29/10, com o cliente no controle.

S8-03 Ações de mundo no catálogo

UC-H05 UC-H07 · depende de S9-03

O mestre passa a mudar o mundo: vida, munição, personagens e itens, com volta e proibições. Entra depois do catálogo de capacidades do S9-03, para que cada ação venha do mapa, não de tentativa e erro.

- Declaração de ações no perfil: operação, faixa por intensidade, duração, volta, proibições; a plataforma recusa ação sem volta ou que possa derrotar sozinha.
- Execução pelo executor: ler antes, escrever, reler, registrar, desfazer no prazo pelo relógio de fim; reversão pelo jogador.
- Primeiras ações do jogo de abertura: tirar e devolver vida dentro de faixa segura, tirar munição; ações sobre entidade (vida de um inimigo) quando a lista estiver validada.

CRITÉRIO DE ACEITE

Dado uma situação de combate, **quando** o mestre decide baixar a vida do jogador até o mínimo da faixa por 20 s, **então** o valor cai, é relido igual, volta sozinho no prazo, nunca abaixo do mínimo, e o registro tem antes, depois, motivo e volta.

S8-04 Ferramenta do mapeador para a memória

UC-H01 UC-H03 UC-H06

O que o S8-00 fez à mão vira ferramenta repetível para qualquer jogo.

- Roteiro guiado: o que fazer no jogo entre fotos, com pausa e foto pelo canal a cada parada.
- Candidatos mostrados foto a foto, com busca por valor, agrupamento, cadeia de ponteiros e projeção; o mapeador confirma, nomeia e grava.
- Publicar e importar perfil com validação local obrigatória.

CRITÉRIO DE ACEITE

Dado um segundo jogo do cliente, **quando** o mapeador segue o roteiro, **então** sai um perfil com pelo menos a câmera e a posição do jogador validados por projeção, sem ninguém editar endereço à mão.

Etapa S9 · mapa completo do jogo de abertura DC-23 · entra antes de S8-03

Decisão do cliente em 29 de outubro: mapear o jogo inteiro de forma sistemática, não sob demanda. A reversa sob demanda custou um ciclo por pergunta e deixou buracos que travaram o jogo (a ligação de módulos não tinha sido lida antes de usá-la). O mapa é pago uma vez e consultado; as ações de mundo saem dele.

S9-01 Decompilação total do executável e dos módulos da ISO

UC-H03

Tudo em C legível, com os nomes do próprio jogo, antes de qualquer pergunta nova.

- Todas as funções do executável e dos módulos carregados já decompiladas (10.673, sem falhas, fora do repositório).
- Realocador próprio do formato SNR2 em Python, a partir do ligador do jogo lido no S8-02: cada módulo da ISO (todos os inimigos e personagens) realocado para um endereço virtual e decompilado com símbolos, sem precisar do jogo rodando.
- Índice versionado no repositório: endereço, nome, subsistema, tamanho, chamadores, nota; agrupamento por classe e por módulo.

CRITÉRIO DE ACEITE

Dado a ISO e o executável, **quando** o roteiro roda do zero, **então** existe um arquivo C por função do executável e de cada módulo da ISO, e o índice lista todas com subsistema; uma função pedida por nome é achada no índice sem abrir o Ghidra.

S9-02 Estruturas e tabelas do jogo

UC-H01 UC-I01

O que cada deslocamento significa, numa fonte só.

- Bloco global do jogador, personagem (cEm), item e maleta, tarefas e threads, módulos e ligador, listas de sala (.esl), arquivos do AFS, tabelas de armas e de peças, sistema de cenas (evm, SceAt*).
- Cada estrutura com deslocamentos, tipos, quem escreve e quem lê, e a prova (vigia, decompilado ou foto).
- Entra no perfil de memória do jogo e num documento de estruturas no repositório.

CRITÉRIO DE ACEITE

Dado uma estrutura do documento, **quando** um campo é lido pelo canal no jogo rodando, **então** o valor tem o significado descrito; três campos sorteados por estrutura conferidos ao vivo pelo cliente.

S9-03 Catálogo de capacidades do jogo

UC-H05 UC-H07

O que o mestre pode chamar com segurança, com pré-condições e o que trava. É a matéria-prima do S8-03.

- Lista das funções chamáveis pela chamada remota: criar personagem, dar item, dinheiro, vida, teleporte, cena, destruir, e as que não podem (dormem pelo escalonador).
- Para cada uma: argumentos, pré-condições (módulo residente, tipo que obedece posição), efeito visível, como desfazer, risco.
- Testes contra o simulador para a forma das chamadas e prova real por capacidade, com o cliente no controle.

CRITÉRIO DE ACEITE

Dado o catálogo, **quando** o cliente escolhe três capacidades ao acaso, **então** cada uma funciona ao vivo como descrita e o jogo segue; o que está marcado como "trava" não entra no catálogo de ações.

S9-04 Roteiro reutilizável por jogo

UC-H06

O próximo jogo começa pelo que já funcionou, na ordem em que funcionou.

- Sequência: identificar o jogo, extrair o executável, achar símbolos, foto da memória, projeto do Ghidra, canal, vigia, chamada remota, módulos; o que é universal e o que é do jogo de abertura.
- Cada passo com a ferramenta da bancada que o executa e o que registrar no perfil.
- Primeira execução de ponta a ponta no segundo jogo do cliente.

CRITÉRIO DE ACEITE

Dado um segundo jogo, **quando** o roteiro é seguido do zero, **então** em cada passo se sabe o que fazer e o que esperar, e o que não se aplica está dito; o resultado é um perfil inicial e um mapa de funções do segundo jogo.

CAPÍTULO 9

Ambientes, testes e definição de pronto

Três ambientes, cinco níveis de teste e uma lista que vale para todo card. Nada aqui é opcional: é o que evita que a primeira sessão jogada seja também o primeiro teste.

AMBIENTE	ONDE RODA	COM O QUE CONVERSA	O QUE PROVA
dev	Máquina do cliente	Emulador real, serviço real com chave local	O ciclo de verdade, as provas de captura e de slot, as sessões jogadas.
CI	GitHub Actions	Simulador do canal; respostas gravadas do serviço; sem chave, sem jogo, sem BIOS	Plataforma, catálogo, máquina de estados, registro, cenários de falha.
bancada	Máquina do cliente, sob demanda	Serviço real, dois adaptadores	Qualidade de decisão por regra, custo e latência, escolha do serviço.

Os cinco níveis de teste

NÍVEL	ONDE	O QUE COBRE	META
Unitário	`sessao`, `catalogo`, `mestre`	Máquina de estados, validação dupla, faixas, intervalo, memória curta. Sem rede, sem emulador.	Toda transição da tabela, inclusive as inválidas; todo caminho de descarte.
Registro	`registro`	Esquema, gatilhos, cadeia ciclo a intervenção, reprodução.	As três tentativas proibidas falham; uma sessão gravada é lida inteira.
Simulador	`simulador`	Ponte contra o canal falso em modo original e próprio; cenários de falha; vigia de velocidade.	Nenhum cenário pode regredir sem quebrar o build.
Contrato	`servicos`	Respostas gravadas, válidas e inválidas, contra os contratos 1 e 2.	Toda resposta inválida conhecida é recusada; nenhuma válida é recusada.
Jogado	`emulador real`	Sessão do cliente com o jogo de abertura; registro de defeitos.	Toda intervenção registrada; nenhuma fora do catálogo; o cliente aprova.

Definição de pronto, válida para todo card

- O código passou no lint, nos tipos e em todos os testes do CI.
- O critério de aceite do card foi executado por outra pessoa, não só pelo autor; neste projeto, pelo cliente quando envolve jogar.
- Todo caminho de exceção do caso de uso ligado ao card tem teste.
- Nenhum comando chega ao emulador fora da execução validada; nenhuma linha de registro é alterada ou apagada.
- Nenhuma chave, captura de jogo, BIOS ou imagem de jogo entrou no repositório.
- Log estruturado com sessão e ciclo em toda operação que muda estado.

O simulador é a peça que economiza mais tempo. O emulador real precisa de janela, jogo e atenção; o serviço custa dinheiro a cada chamada. O card S0-03 entrega um canal falso com o mesmo protocolo e os cenários de falha; com ele, as etapas S1 a S3 e S5 inteiras rodam no CI, e a versão própria de S4 nasce com os comandos já testados do lado da plataforma.

FECHAMENTO

Por onde começar

A sequência dos primeiros cards, sem ambiguidade.

PASSO 1

Repositório, sessão viva e simulador

Cards S0-01 a S0-04. No fim, qualquer pessoa clona, roda um comando, e o CI prova o registro e o canal falso.

PASSO 2

Fazer a ponte falar

Cards S1-01 a S1-03, contra o emulador real do cliente e contra o simulador. No fim, um estado reservado salvo e recarregado pelo terminal.

PASSO 3

Dar olhos ao mestre

Cards S2-01 a S2-03. No fim, uma captura do jogo de abertura vira uma situação validada.

PASSO 4

O esqueleto

Cards S3-01 a S3-06. No fim, o cliente joga. Só depois disso alguém compila o emulador.

O que este documento decide e fecha. DC-15 (slots 90 a 99, fila de 10, provado), DC-16 (formato fechado da situação, contrato 1), DC-17 (tons e intensidades), DC-18 (teclas), DC-19 (texto do aviso), LC-11 (confirmação pelo arquivo, com número), LC-12 (vigia dentro do emulador). Todas sobem ao Mapa do Projeto como revisão 4 junto com a aprovação deste documento. **Versão 1.1:** detalha a fase 4 decidida no Mapa (revisão 6, DC-20 e DC-21) sem fechar nada novo; LC-14 a LC-16 ficam com o card S7-00.

O que este documento assume e precisa ser confirmado.

- Que a receita de compilação do projeto roda na máquina do cliente com as ferramentas já instaladas. O card S4-01 mede espaço e tempo de máquina e grava o que faltou (lacuna LC-10); disco deixou de ser risco com 87 GB livres.
- Que a captura pela janela funciona também com o jogo em tela cheia sem borda. A prova foi em janela; o card S2-01 testa a outra (lacuna LC-13).
- Que as teclas Ctrl+Alt+Z, P e Q não colidem com nenhum atalho do emulador nem do jogo. Confirmar no esqueleto.
- Qual serviço de linguagem fica. O primeiro adaptador é o do provedor cujo acesso o cliente já tem; a escolha é da bancada, por número.
- Que o levantamento do que já existe (lacuna LC-07) continua sem fazer, e fica antes da fase 2.

Quando consideramos a fase 1 entregue. Quando o cliente joga uma sessão inteira do jogo de abertura, em modo completo, com toda intervenção registrada com antes, depois e motivo, nenhuma fora do catálogo, e diz se o jogo deixou de ser o de sempre. Esse é o único critério que importa: não é o código pronto, é o mestre sendo sentido.